

Université Abdelmalek Essaâdi

École Nationale des Sciences Appliquées – Al Hoceima

COMPTE RENDU

TP N°2 : J2EE / Java EE (JEE) ou Jakarta EE

Manipulation de base des Servlets et JSP

Filière :

TDIA2 – Semestre 4

Réalisé par :

HAITI Aya

Année universitaire :

2025 / 2026

Encadré par :

Pr. Mohamed CHERRADI

1. Introduction

Dans ce TP, nous avons étudié les technologies Servlets et JSP dans le cadre du développement des applications web dynamiques en Java EE. L'objectif est de comprendre comment créer une application web simple permettant la gestion des clients et des réservations. Ce travail consiste à développer une application conforme aux étapes et captures fournies dans l'énoncé, en respectant l'architecture Java EE basée sur le modèle MVC (Modèle-Vue-Contrôleur).

2. Objectifs du TP

- Comprendre le rôle des Servlets dans le traitement des requêtes HTTP
- Utiliser les JSP (JavaServer Pages) pour l'affichage côté client
- Manipuler les données entre les différentes couches de l'application
- Développer une application web dynamique simple avec validation des formulaires
- Maîtriser la configuration d'Apache Tomcat comme serveur d'application

3. Description de l'Application

L'application développée est un système de gestion des clients et des réservations pour un hôtel. Elle permet les opérations suivantes :

3.1 Données manipulées

Entité	Champs
Client	nom, prénom, téléphone, e-mail
Réservation	nom, prénom, téléphone, e-mail + type, prix, vue

4. Architecture du Projet

Le projet suit une architecture MVC (Modèle-Vue-Contrôleur) et est structuré comme suit :

□ JSP (Vues)

```
Inscription.jsp - formulaire  
d'ajout client  
  
infoClient.jsp - affichage des  
données client  
  
Reservation.jsp - formulaire  
d'ajout réservation  
  
infoReservation.jsp - affichage  
des réservations  
  
index.jsp - page d'accueil
```

□ Servlets (Contrôleurs)

```
creerClient.java - traitement  
client (/addClt)  
  
creerReservation.java -  
traitement réservation (/addRsv)
```

5. Fonctionnement de l'Application

5.1 Partie Client

URL de la Servlet : @WebServlet("/addClt")

Le flux de traitement pour la gestion des clients est le suivant :

- L'utilisateur accède au formulaire Inscription.jsp
- Il saisit ses informations (nom, prénom, téléphone, e-mail)
- Les données sont envoyées via POST à la Servlet creerClient (/addClt)
- La Servlet valide que tous les champs sont remplis
- En cas d'erreur : redirection vers Inscription.jsp avec message d'erreur
- En cas de succès : forward vers infoClient.jsp avec les données en attributs de requête

5.2 Partie Réservation

URL de la Servlet : @WebServlet("/addRsv")

Le flux de traitement pour la gestion des réservations est le suivant :

- L'utilisateur accède au formulaire Reservation.jsp
- Il saisit les données client + type de chambre, prix et vue
- Les données sont envoyées via POST à la Servlet creerReservation (/addRsv)
- La Servlet effectue la validation de tous les champs obligatoires
- En cas d'erreur : redirection vers Reservation.jsp avec message d'erreur
- En cas de succès : forward vers infoReservation.jsp avec toutes les données

6. Code Source

6.1 Servlet creerClient.java

```
@WebServlet("/addClt")
public class creerClient extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        req.setCharacterEncoding("UTF-8");

        String nom      = req.getParameter("nom");
        String prenom    = req.getParameter("prenom");
        String telephone = req.getParameter("telephone");
        String email     = req.getParameter("email");

        // Validation : tous les champs doivent être remplis
        if (nom == null || nom.trim().isEmpty() || ...) {
            req.setAttribute("erreur", "Vous devez remplir tous les champs");
            req.getRequestDispatcher("Inscription.jsp").forward(req, resp);
            return;
        }

        // Transmettre les données à infoClient.jsp
        req.setAttribute("nom", nom.trim());
```

```

        req.setAttribute("prenom", prenom.trim());
        req.setAttribute("telephone", telephone.trim());
        req.setAttribute("email", email.trim());

        req.getRequestDispatcher("infoClient.jsp").forward(req, resp);
    }
}

```

6.2 Servlet creerReservation.java

```

@WebServlet("/addRsv")
public class creerReservation extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        req.setCharacterEncoding("UTF-8");

        String nom = req.getParameter("nom");
        String prenom = req.getParameter("prenom");
        String type = req.getParameter("type");
        String prix = req.getParameter("prix");
        String vue = req.getParameter("vue");
        // ... (autres champs)

        // Validation et forward vers infoReservation.jsp
        req.setAttribute("type", type);
        req.setAttribute("prix", prix.trim());
        req.setAttribute("vue", vue);
        req.getRequestDispatcher("infoReservation.jsp").forward(req, resp);
    }
}

```

6.3 Inscription.jsp (formulaire client)

```

<%@ page contentType="text/html; charset=UTF-8" %>
<form action="addClt" method="post">
    <label>Nom</label>
    <input type="text" name="nom" />

    <label>Prénom</label>
    <input type="text" name="prenom" />

    <label>Téléphone</label>
    <input type="text" name="telephone" />

    <label>E-mail</label>
    <input type="email" name="email" />

    <button type="submit">Ajouter Client</button>
</form>

```

6.4 infoClient.jsp (affichage client)

```

<%@ page contentType="text/html; charset=UTF-8" %>
<p class="success">Client ajoutée avec succès</p>
<table>
    <tr>
        <th>Nom</th><th>Prénom</th>
        <th>Téléphone</th><th>E-mail</th>
    </tr>
    <tr>

```

```

        <td>${nom}</td><td>${prenom}</td>
        <td>${telephone}</td><td>${email}</td>
    </tr>
</table>

```

6.5 Reservation.jsp (formulaire réservation)

```

<%@ page contentType="text/html; charset=UTF-8" %>
<form action="addRsv" method="post">
    <!-- Champs client : nom, prenom, telephone, email -->
    <label>Type</label>
    <select name="type">
        <option value="single">chambre single</option>
        <option value="double">chambre double</option>
        <option value="suite">suite</option>
    </select>

    <label>Prix</label>
    <input type="number" name="prix" step="0.1" min="0" />

    <label>Vue</label>
    <select name="vue">
        <option value="piscine">piscine</option>
        <option value="mer">mer</option>
        <option value="jardin">jardin</option>
    </select>
    <button type="submit">Ajouter Réservation</button>
</form>

```

7. Technologies Utilisées

Technologie	Rôle
Java EE (Servlets)	Traitement des requêtes HTTP, logique métier, validation
JSP (JavaServer Pages)	Affichage dynamique des données côté client
HTML / CSS	Structure et mise en forme des pages web
Apache Tomcat	Serveur d'application Java EE
Maven (pom.xml)	Gestion des dépendances du projet
Jakarta EE API	API Jakarta Servlet 6.0

8. Résultats Obtenus

✓ Application fonctionnelle déployée sur Apache Tomcat

- Ajout des clients réussi avec validation des champs obligatoires
- Ajout des réservations réussi avec sélection du type, prix et vue
- Affichage correct des données dans les pages JSP correspondantes
- Bonne communication entre les Servlets et les JSP via les attributs de requête
- Gestion des erreurs de saisie avec messages d'erreur affichés à l'utilisateur
- Encodage UTF-8 correctement configuré pour les caractères spéciaux

9. Difficultés Rencontrées

Difficulté	Solution apportée
Compréhension du fonctionnement des Servlets	Étude de la documentation Jakarta EE et des exemples fournis
Transmission des données entre Servlet et JSP	Utilisation de <code>req.setAttribute()</code> / <code>\${attribut}</code> en JSP
Gestion des erreurs de saisie	Validation côté Servlet avec redirection et attribut erreur
Configuration de l'environnement Tomcat	Configuration du <code>pom.xml</code> avec les dépendances Jakarta Servlet

10. Conclusion

Ce TP nous a permis de découvrir les bases du développement web avec Java EE / Jakarta EE. Nous avons appris à utiliser les Servlets pour le traitement des données côté serveur et les JSP pour l'affichage dynamique côté client.

L'implémentation de cette application de gestion des clients et des réservations nous a permis de comprendre le modèle MVC et la communication entre les différentes couches d'une application web Java.

Cela constitue une base essentielle pour développer des applications web plus avancées intégrant des bases de données, une authentification et une architecture multicouche.

11. Améliorations Possibles

- Intégration d'une base de données MySQL avec JDBC pour la persistance des données
- Ajout des fonctionnalités de modification et suppression des enregistrements
- Amélioration de l'interface graphique avec Bootstrap ou un framework CSS
- Ajout d'une validation avancée des formulaires côté client (JavaScript)
- Implémentation d'un système d'authentification et de gestion des sessions
- Migration vers une architecture REST avec des API JSON